

Investigate inode structures using commands such as `ls -li`, `stat`, and `find`. Create hard links and symbolic links and explain their impact on inode allocation.

Design a program using `mmap()` to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

```
[varnikakomali@VARNIKAS-MacBook-Air Practical % nano practical10.c
[varnikakomali@VARNIKAS-MacBook-Air Practical % gcc practical10.c -o practical10 && ./practical10
=== Part 1: Inodes, Hard Links, and Symbolic Links ===

1. Original file created:
orig_file.txt | Inode: 22717376 | Links: 1 | Size: 39 bytes

2. Hard link created (points to identical inode, link count increases):
orig_file.txt | Inode: 22717376 | Links: 2 | Size: 39 bytes
hard_link.txt | Inode: 22717376 | Links: 2 | Size: 39 bytes

3. Symbolic link created (allocates a new inode pointing to pathname):
orig_file.txt | Inode: 22717376 | Links: 2 | Size: 39 bytes
hard_link.txt | Inode: 22717376 | Links: 2 | Size: 39 bytes
soft_link.txt | Inode: 22717379 | Links: 1 | Size: 13 bytes

Cleaned up link files.

=== Part 2: Memory-Mapped File I/O (mmap) ===

Data written to mmap region: "Written directly into memory via mmap()!"
Read back from disk file: "Written directly into memory via mmap()!"
varnikakomali@VARNIKAS-MacBook-Air Practical %
```